```
Ext.define('Hammerhead.grid.receiving.BoxAttributeGrid', {
    extend: 'Hammerhead.grid.CellField',
    alias: 'widget.boxattributegrid',
    requires: ['Hammerhead.store.receiving.BoxAttributes',
                'Hammerhead.view.customcontrols.CubiscanField'
        ],
    allowTypeSelection: true,
    allowIndicators: false,
    invalidRefNum: 'Box Number cannot be blank.',
    invalidCls : Ext.baseCSSPrefix + 'form-invalid',
    defaultType: 1,
    boxNum: 0,
    allowBlankColumns: false,
    reqMark: '<span class="requiredMark">*</span>',
    features:[{
        ftype:'summary' ,
        dock:'bottom'
     }],
    viewConfig: {
        deferEmptyText: false,
        emptyText: '<div class="x-grid-empty">No Box Attributes to Display</div>'
    },
    allowAddRemove:true,
    gridUpdated: false,
    initComponent: function() {
        var me = this, hasIndicatorNumber = false;
        me.columns = [{
            header: 'Box #',
            dataIndex: 'boxId',
            editor: {
                xtype: 'numberfield',
                allowDecimals: false,
                allowNegative: false
            },
            summaryType: 'count',
            summaryRenderer: function(value) {
                return 'Total Boxes: ' + value;
            }
        },{
                header: 'Type',
                dataIndex: 'boxType',
                editor: {
                                xtype: 'combo',
                    displayField: 'displayField',
                    valueField: 'value',
                                store: Ext.create('Ext.data.ArrayStore',{
                                        fields: ['value', 'displayField'],
                                        data: [['BOX/CASE', 'BOX/CASE'],['CRATE', 'CRA
TE'],['ENVELOPE', 'ENVELOPE'],['OVERSIZED', 'OVERSIZED'],['SKID', 'SKID']]
                                })),
                                typeAhead: false,
                                allowBlank: true,
                                editable: false,
                                triggerAction: 'all',
                                mode: 'local',
                                forceSelection: true,
                                emptyText: 'Select Box Type'
                }
        },{
            header: 'Length (in)',
            dataIndex: 'length',
            editor: {
                xtype: 'numberfield',
                allowBlank: me.allowBlankColumns,
                minValue: 1,
```

CANCELED

```
                allowDecimals: false,
                allowNegative: false
            },
                        renderer: function(value) {
                                return Ext.util.Format.number(value || 0, '0');
                        },
            summaryType: 'sum',
            summaryRenderer: function(value) {
                return 'Total Length (in): ' + Ext.util.Format.number(value || 0, '0')
;
            }
        },{
            header: 'Width (in)',
            dataIndex: 'width',
            editor: {
                xtype: 'numberfield',
                allowBlank: me.allowBlankColumns,
                minValue: 1,
                allowDecimals: false,
                allowNegative: false
            },
                        renderer: function(value) {
                                                        return Ext.util.Format.number(
value || 0, '0');
                        },
            summaryType: 'sum',
            summaryRenderer: function(value) {
                return 'Total Width (in): ' + Ext.util.Format.number(value || 0, '0');
            }
        },{
            header: 'Height (in)',
            dataIndex: 'height',
            editor: {
                xtype: 'numberfield',
                allowBlank: me.allowBlankColumns,
                minValue: 1,
                allowDecimals: false,
                allowNegative: false
            },
                        renderer: function(value) {
                                return Ext.util.Format.number(value || 0, '0');
                        },
            summaryType: 'sum',
            summaryRenderer: function(value) {
                return 'Total Height (in): ' + Ext.util.Format.number(value || 0, '0')
;
            }
        },{
            header: 'Weight (lb)',
            dataIndex: 'weight',
            editor: {
                xtype: 'numberfield',
                allowBlank: me.allowBlankColumns,
                minValue: 0,
                allowDecimals: false,
                allowNegative: false
            },
                        renderer: function(value) {
                                return Ext.util.Format.number(value || 0, '0');
                        },
            summaryType: 'sum',
            summaryRenderer: function(value) {
                return 'Total Weight (lb): ' + Ext.util.Format.number(value || 0, '0')
;
            }
```

```
        }];
        me.selModel = Ext.create('Ext.selection.CheckboxModel', {
                checkOnly : true,
                allowDeselect : true,
                mode : 'MULTI'
        });
        if (!me.store ) {
                me.store = Ext.create('Hammerhead.store.receiving.BoxAttributes');
                if(me.initialConfig.data) {
                        me.store.add(me.initialConfig.data);
                }
        }

                 me.extraButtons=['-',{
                 text: 'Add Multiple',
                 tooltip:'Add multiple boxes of the same size',
                 handler: function() {
                        me.onSelectMultiple();
                 },
                 scope: me
             }, {
                                xtype: 'cubiscanfield',
                                name: 'cubiscanLocation',
                                itemId: 'cubiscanField',
                                disabled: true,
                                fieldDefaults: {
                                    labelWidth : 150,
                                    width : 200,
                                    allowBlank : true
                                },
                                fieldLabel: '',
                                width : 500,
                                hiddenFields: [],
//                              disableFields: [0,1],
                                listeners: {
                                        cubiscanselected: function(field, rec) {
                                                var form = field.up('form').getForm();
                                                form.findField('cubiscanIp').setValue(
rec.get('ipAddress'));
                                        },
                                        measurementreceived: function(field, payload)
{
                                                var measurement = payload && (payload.
measurement || payload);
                                                if (!measurement) {
                                                        Ext.Msg.alert('No Data', 'No m
easurement data returned.');
                                                        return;
                                                }
                                                if (measurement && measurement.dimensi
onal_units !== 'in') {
                                                        Ext.Msg.alert('Cubiscan', 'Cub
iscan dimensions need to be set to "in" and weight set to "lb"');
                                                        return;
                                                }
                                            me.updateWithCubiscanData(measurement);
                                                },
                                        measurementerror: function(field, errMsg) {
                                                Ext.Msg.alert('Measurement Error', err
Msg);
                                        }
                                }
                }
        ];
```

```
        me.listeners = Ext.apply(me.listeners || {}, {
                afterrender: function(grid){
                        var sm = grid.getSelectionModel(), store = grid.getStore();
                        if (sm && !sm.hasListener('selectionchange')) {
                                sm.on('selectionchange', grid.onSelectionChange, grid)
;
                        }

                        if (store) {
                                store.on('load', function() {
                                        grid.onSelectionChange(sm, sm.getSelecton ? sm
.getSelection() : []);
                                }, grid, {single: false});
                                store.on('datachanged', function() {
                                        grid.onSelectionChange(sm, sm.getSelecton ? sm
.getSelection() : []);
                                }, grid, {single: false});

                        }
                        Ext.defer(function(){
                                grid.onSelectionChange(sm, sm.getSelecton ? sm.getSele
ction() : []);
                        },0 );
                }
        });


        me.callParent(arguments);
    },
    refreshData: function(referenceNumberModels) {
        var me = this;

        me.store.removeAll();
        me.store.add(referenceNumberModels);
    },
    onAddButton: function() {
        var me = this, rowEditor = me.getRowEditor(),
            r = Ext.create('Hammerhead.model.receiving.BoxAttribute', {
                boxId: me.store.getCount()+1,
                boxType: '',
                length: '',
                width: '',
                height: '',
                weight: ''
        });
        me.store.insert(0, r);
        rowEditor.startEditByPosition({row: 0, column: 0});
    },

    onBeforeEdit: function(roweditng, e) {
        var me = this, allowEdit = true, typeId;
        if(!me.gridUpdated){
                me.gridUpdated = true;
        }
        if(!me.editable) {
            allowEdit = false;
        }
        return allowEdit;
    },
    onCancelEdit: function(plugin, e) {
        var me=this;
        me.callParent(arguments);
        me.isValid();
    },
```

```
    onEdit: function(cellEditing, context) {
        var me=this;
        me.isValid();
        me.callParent(arguments);
    },
    onRemoveButton: function() {
        var me = this;
        me.callParent();
        me.isValid();
        me.getView().refresh();
    },
    onSelectionChange: function(sm, recs) {
        var me = this, button = me.query('button')[1], i, typeId, cubiscan = me.down('
#cubiscanField');
        for(i=0; i<recs.length; i++) {
        }
        button.setDisabled(sm.getCount() < 1);
                cubiscan.setDisabled(sm.getCount() < 1);
                if (sm.getCount() < 1) {
                        if (cubiscan.getEl()) {
                                cubiscan.getEl().setStyle('pointer-events', 'none');
                        }
                } else {
                        if (cubiscan.getEl()) {
                                cubiscan.getEl().setStyle('pointer-events', '');
                        }
                }
    },
    isValid : function() {
        var me = this, valid = me.validateValue();
        return valid;
    },
    validateValue: function() {
        var me = this,
            errors = me.getErrors(),
            isValid = Ext.isEmpty(errors);
        if (!me.preventMark) {
            if (isValid) {
                me.clearInvalid();
                me.setActiveErrors(null);
                me.fireEvent('validitychange', me, null);
            } else {
                if (isValid !== me.wasValid) {
                    me.markInvalid(errors);
                    me.setActiveErrors(Ext.Array.from(errors));
                    me.fireEvent('validitychange', me, me.activeError);
                }
            }
        }
        me.wasValid = isValid;
        return isValid;
    },
    getErrors: function() {
        var me = this, errors = [];
        if(me.allowBlank) {
            return errors;
        }
        me.store.each(function(r) {
            return errors;
        });
        return errors;
    },
    markInvalid : function(errors) {
        this.setActiveErrors(Ext.Array.from(errors));
    },
```

```
    clearInvalid : function() {
        this.unsetActiveError();
    },
    hasActiveError: function() {
        return !!this.getActiveError();
    },
    getActiveError : function() {
        return this.activeError || '';
    },
    setActiveErrors: function(errors) {
        var me = this;
        me.activeErrors = errors;
        if (Ext.isEmpty(errors)){
            me.activeError ='';
        } else {
            me.activeError = errors.length == 1 ? errors[0] : '';
        }
        me.renderActiveError();
    },
    unsetActiveError: function() {
        delete this.activeError;
        delete this.activeErrors;
        this.renderActiveError();
    },
    renderActiveError: function() {
        var me = this,
            activeError = me.getActiveError(),
            hasError = !!activeError;

        if (activeError !== me.lastActiveError) {
            me.fireEvent('errorchange', me, activeError);
            me.lastActiveError = activeError;
        }

        if (me.rendered && !me.isDestroyed && !me.preventMark) {
            me.el[hasError ? 'addCls' : 'removeCls'](me.invalidCls);

            me.getActionEl().dom.setAttribute('aria-invalid', hasError);
        }
    },
    onSelectMultiple: function() {
        var rec = Ext.create('Hammerhead.model.receiving.BoxAttribute');
        this.edit(rec);
    },
    edit: function(rec) {
        var me = this, win = Ext.create('Hammerhead.window.receiving.BoxMultiple', {re
c: rec});
        win.on('update', me.update, me);
        win.show();
    },
    update: function(w, r) {
        var me = this, rowEditor = me.getRowEditor();

        for(i=0; i<r.data.pieces; i++) {
            rr = Ext.create('Hammerhead.model.receiving.BoxAttribute', {
                boxId: me.store.getCount()+1,
                boxType: r.data.boxType,
                length: r.data.length,
                width: r.data.width,
                height: r.data.height,
                weight: r.data.weight
                });
            me.store.insert(0, rr);
            rowEditor.startEditByPosition({row: 0, column: 0});
        }
```

```
            w.close();
    },
    getTotalBoxes: function() {
        var me = this;
        return me.store.getCount();
    },
        updateWithCubiscanData: function(measurement) {
                var me = this, s = me.getSelectionModel().getSelection();
                        Ext.suspendLayouts();
                        Ext.Array.each(s, function(r) {

                                //check if already populated like in packing?
                                r.set('length', Math.round(Number(measurement.length)
|| 0 ));
                                r.set('width', Math.round(Number(measurement.width) ||
 0 ));
                                r.set('height', Math.round(Number(measurement.height)
|| 0 ));
                                r.set('weight', Math.round(Number(measurement.weight)
|| 0 ));

                        });
                        Ext.resumeLayouts(true);

        }
});
```